Competency Level 9.10: Uses data structures in programs

Learning Outcomes:

- Briefly explains the use of data structures
- Uses relevant data structures in programming

Data structures:

- Strings
- Lists
- Tuples
- Dictionaries

List

A list is a collection, which is ordered and changeable. In Python, lists are written with square brackets.

Example 25:

Create List

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
print(mylist)
```

Output

```
>>>
['cat', 'dog', 'fish']
>>>
```

Example 26:

Print the second item of the list:

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
print(mylist[1])
```

Output

```
>>>
dog
>>>
```

Example 27:

Change the second item as an **ant**

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
mylist[1] = "ant"

print(mylist)
```

Output

```
>>>
['cat', 'ant', 'fish']
>>>
```

Example 28:

Print all items in the list, one by one:

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
for b in mylist:
  print(b)
```

Output

```
>>>
cat
dog
fish
>>>
```

Example 29:
Check if "cat" is present in the list

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
if "cat" in mylist:
  print("Yes, 'cat' is in my list")
```

Output

```
>>>
Yes, 'cat' is in my list
>>>
```

Example 30:
Print the total number of items in the list:

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
print(len(mylist))
```

Output

```
>>>
3
>>>
```

Add an item to the list
The append() method uses to append an item in a list

Example 31:
Appending ant to the list

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
mylist.append("ant")
print(mylist)
```

Output

```
>>>
['cat', 'dog', 'fish', 'ant']
>>>
```

Inserting items in the middle of a list
Example 32:
Insert ant in the second position in the list

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
mylist.insert(1, "ant")
print(mylist)
```

Output

```
>>>
['cat', 'ant', 'dog', 'fish']
>>>
```

Removing Items from a list
The remove() method removes the specific item:

Example 33:
Removing the **dog** from the list

Code

```
File Edit Format Run Options Windows Help
mylist = ["cat", "dog", "fish"]
mylist.remove("dog")
print(mylist)
```

Output

```
>>>
['cat', 'fish']
>>>
```

Removing the specified item from the list
The **del** keyword removes the specified indexed item:

Example 34:
Removing **zero-indexed** item from the list, mylist

Code

```
mylist = ["cat", "dog", "fish"]
del mylist[0]
print(mylist)
```

Output

```
>>>
['dog', 'fish']
>>>
```

Clearing the list
The clear() method empties the list:

Example 35:
To clear all the items from mylist

Code

```
mylist = ["cat", "dog", "fish"]
mylist.clear()
print(mylist)
```

Output

```
>>>
[]
>>>
```

Python has a set of built-in methods that you can use on lists.

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Note: Some methods already discussed above

Tuples

A tuple is a collection which is ordered and **unchangeable (immutable)**. In Python, tuples are written with round brackets.

Example 36:

Create a Tuple

Code

```
File Edit Format Run Options Windows Help
cityname = ("Kandy", "Jaffna", "Colombo")
print(cityname)
```

Output

```
>>>
('Kandy', 'Jaffna', 'Colombo')
>>>
```

Example 37:

Return the item in position 1:

Code

```
File Edit Format Run Options Windows Help
cityname = ("Kandy", "Jaffna", "Colombo")
print(cityname[1])
```

Output

```
>>>
Jaffna
>>>
```

Example 38:

As discussed earlier values in a tuple cannot be changed

Code

```
File Edit Format Run Options Windows Help
cityname = ("Kandy", "Jaffna", "Colombo")

cityname[1] = "Galle"
# the value is still the same:
print(cityname)
```

Output

```
>>>
Traceback (most recent call last):
  File "C:\Python33\python program\Pyhon programme\ex39.py"
, line 3, in <module>
    cityname[1] = "Galle"
TypeError: 'tuple' object does not support item assignment
>>>
```

Example 39:

Iterate items and print the values:

Code

```
File Edit Format Run Options Windows Help
subjectTuple =("Maths", "Science", "ICT")
for x in subjectTuple:
  print(x)
```

Output

```
>>>
Maths
Science
ICT
>>>
```

Example 40:

Checking the availability of an item in a tuple

Check if "ICT" is present in the tuple:

Code

```
File Edit Format Run Options Windows Help
subjectTuple =("Maths", "Science", "ICT")
if "ICT" in subjectTuple:
  print("Yes, 'ICT' is in the fruits tuple")
```

Output

```
>>>
Yes, 'ICT' is in the fruits tuple
>>>
```

Example 41:
Print the number of items in the tuple:
Code Output

```
File Edit Format Run Options Windows Help
subjectTuple =("Maths", "Science", "ICT")
print(len(subjectTuple))
```

```
>>>
3
>>>
```

Tuples:
Tuples are immutable, therefore, cannot remove items, but can delete the tuple.

Built-in methods in tuples

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

Dictionaries
A dictionary is a collection that is unordered, changeable and indexed. In Python, dictionaries are written with curly brackets, and they have keys and values.
Example 42:
Create and print a dictionary
Code

```
File Edit Format Run Options Windows Help
Stu_details =    {
  "Name": "Saman",
  "Country": "Sri  Lanka",
  "year": 2019
}
print(Stu_details)
```

Output

```
{'Name': 'Saman', 'country': 'Sri Lanka', 'Year': 2019}
>>>
```

Get the value of a key.
get() method can be also used to get the value of the key.
Example 43:
Get the value of the "Country" key
CodeOutput

```
File Edit Format Run Options Windows Help
Stu_details =    {
  "Name": "Saman",
  "Country": "Sri  Lanka",
  "year": 2019
}
x = Stu_details["Country"]
print(x)
x = Stu_details.get("Country")
print(x)
```

```
>>>
Sri  Lanka
Sri  Lanka
>>>
```

Changing the values of the keys:
Example 44:
Change the "year" value from 2019 to 2018:

Code

```
File  Edit  Format  Run  Options  Windows  Help
Stu_details =    {
  "Name": "Saman",
  "Country": "Sri  Lanka",
  "year": 2019
}

Stu_details["year"] = 2018

print(Stu_details)
```

Output

```
{'Name': 'Saman', 'Country': 'Sri Lanka', 'year': 2018}
>>>
```

Printing the key names.
Example 45:
Printing key names from the Dictionary

Code Output

```
File  Edit  Format  Run  Options  Windows  Help
Stu_details =    {
  "Name": "Saman",
  "Country": "Sri  Lanka",
  "year": 2019
}
for a in Stu_details:
  print(a)
```

```
Name
Country
year
>>>
```

Printing the key values
Example 46:
Printing all key values from a dictionary

Code

```
File  Edit  Format  Run  Options  Windows  Help
Stu_details =    {
  "Name": "Saman",
  "Country": "Sri Lanka",
  "year": 2019
}
for a in Stu_details :
  print(Stu_details[a])
```

Output

```
>>>
==========
Saman
Sri Lanka
2019
>>>
```

You can also use the **values()** function to return the values of a dictionary.
Loop through both keys and values, by using the items() function:

Example 47:

Checking the existing items in a dictionary using the item() function

Code

```
dec.py - D:/nie/pythion/dec.py
File  Edit  Format  Run  Options  Windows  Help
Stu_details={
    "Name": "Saman",
    "Country" :"Srilanka",
    "Year": 2019}
print(Stu_details.values())
Ln: 6 Col: 0
```

Output

```
dict_values(['Saman', 'Sri Lanka', 2019])
>>>
```

Example 48:

Checking whether an existing item in the dictionary

Input

```
File  Edit  Format  Run  Options  Windows  Help
Stu_details =   {
  "Name": "Saman",
  "Country": "Sri Lanka",
  "year": 2019
}
for a in Stu_details:
  print(Stu_details[a])

print("----------------")

for a, b in Stu_details.items():
  print(a,b)
```

Output

```
>>>
============ RESTART: H:
Saman
Sri Lanka
2019
-----------------
Name Saman
Country Sri Lanka
year 2019
>>>
```

Example 49:

Check if "Country" is present in the dictionary

Code

```
*dec2.py - D:/nie/pythion/dec2.py*
File  Edit  Format  Run  Options  Windows  Help
Stu_details={
    "Name": "Saman",
    "Country" :"Srilanka",
    "Year": 2019}
if "Country" in Stu_details:
    print("Yes,' Country' is one of the keys in Stu_details dictionary")
Ln: 10 Col: 0
```

Output

```
Yes, 'Country' is one of the keys in the dictionnary1 dictionary
>>>
```

Example 50:
Print the number of items in the dictionary
Code Output

```
File Edit Format Run Options Windows Help
Stu_details =    {
  "Name": "Saman",
  "Country": "Sri Lanka",
  "year": 2019 }

print(len(Stu_details))
```

```
>>>
======
3
>>>
```

Adding an item to the dictionary, with a new index key and assign a value.
Example 51:
To the school index key, add value 'ABC'in the Stu_details dictionary

Code

```
File Edit Format Run Options Windows Help
Stu_details =    {
  "Name": "Saman",
  "Country": "Sri Lanka",
  "year": 2019 }

Stu_details["School"] = "ABC"

print(Stu_details)
```

Output

```
{'Name': 'Saman', 'Country': 'Sri Lanka', 'year': 2019, 'School': 'ABC'}
>>>
```

Removing Items
There are several methods to remove items from a dictionary:
The **pop()** method removes the item with the specified key name

Example 52:
Removing the country from the dictionary

Code

```
File Edit Format Run Options Windows Help
Stu_details =    {
  "Name": "Saman",
  "Country": "Sri Lanka",
  "year": 2019 }

Stu_details.pop("Country")

print(Stu_details)
```

Output

```
{'Name': 'Saman', 'year': 2019}
>>>
```

The **popitem()** method removes the last inserted item.

Example 53:

Removing the last inserted item

Code

```
File  Edit  Format  Run  Options  Windows  Help
Stu_details =    {
   "Name": "Saman",
   "Country": "Sri Lanka",
   "year": 2019 }

Stu_details.popitem()
print(Stu_details)
```

Output

```
{'Name': 'Saman', 'Country': 'Sri Lanka'}
>>>
```

The **del** keyword removes the item with the specified key name.

Example 54:

Remove country from dictionary

Code

```
File  Edit  Format  Run  Options  Windows  Help
Stu_details =    {
   "Name": "Saman",
   "Country": "Sri Lanka",
   "year": 2019 }

del Stu_details ["Country"]

print(Stu_details)
```

Output

```
{'Name': 'Saman', 'year': 2019}
>>>
```

The **clear()** keyword empties the dictionary.

Example 55:

Clear the dictionary

Code

```
File  Edit  Format  Run  Options  Windows  Help
Stu_details =    {
   "Name": "Saman",
   "Country": "Sri Lanka",
   "year": 2019 }

Stu_details.clear()

print(Stu_details)
```

Output

```
{}
>>>
```

Dictionary Methods
Python has a set of built-in methods that you can use on dictionaries.

Method	Description
clear()	Removes all the elements from the dictionary
fromkeys()	Returns a dictionary with the specified keys and values
get()	Returns the value of the specified key
items()	Returns a list containing the tuple for each key-value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary